

Week 5: Bayesian linear regression and introduction to Stan

06/02/26

Introduction

Today we will be starting off using Stan, looking at the kid's test score data set (available in resources for the [Gelman Hill textbook](#)).

```
library(tidyverse)
library(rstan)
library(tidybayes)
library(here)
```

The data look like this:

```
kidiq <- read_rds(here("data", "kidiq.RDS"))
kidiq
```

```
# A tibble: 434 x 4
  kid_score mom_hs mom_iq mom_age
    <int>   <dbl>   <dbl>   <int>
1      65     1  121.     27
2      98     1   89.4     25
3      85     1  115.     27
4      83     1   99.4     25
5     115     1   92.7     27
6      98     0  108.     18
7      69     1  139.     20
8     106     1  125.     23
9     102     1   81.6     24
```

```
10      95      1   95.1      19
# i 424 more rows
```

As well as the kid's test scores, we have a binary variable indicating whether or not the mother completed high school, the mother's IQ and age.

Descriptives

Question 1

Use plots or tables to show three interesting observations about the data. Remember:

- Explain what your graph/ tables show
- Choose a graph type that's appropriate to the data type

Estimating mean, no covariates

In class we were trying to estimate the mean and standard deviation of the kid's test scores. The `kids2.stan` file contains a Stan model to do this. If you look at it, you will notice the first `data` chunk lists some inputs that we have to define: the outcome variable `y`, number of observations `N`, and the mean and standard deviation of the prior on `mu`. Let's define all these values in a `data` list.

$$y_i | \mu, \sigma \sim N(\mu, \sigma^2)$$

priors:

$$\sigma \sim N^+(0, 10^2)$$

$$\mu \sim N(\mu_0, \sigma_0^2)$$

```
y <- kidiq$kid_score
mu0 <- 80
sigma0 <- 10

# named list to input for stan function
data <- list(y = y,
             N = length(y),
             mu0 = mu0,
             sigma0 = sigma0)
```

Now we can run the model:

```
fit <- stan(file = here("code/models/kids2.stan"),
            data = data,
            # reducing the iterations a bit to speed things up
            chains = 3,
            iter = 500)
```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 2.1e-05 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.21 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 500 [0%] (Warmup)

Chain 1: Iteration: 50 / 500 [10%] (Warmup)

Chain 1: Iteration: 100 / 500 [20%] (Warmup)

Chain 1: Iteration: 150 / 500 [30%] (Warmup)

Chain 1: Iteration: 200 / 500 [40%] (Warmup)

Chain 1: Iteration: 250 / 500 [50%] (Warmup)

Chain 1: Iteration: 251 / 500 [50%] (Sampling)

Chain 1: Iteration: 300 / 500 [60%] (Sampling)

Chain 1: Iteration: 350 / 500 [70%] (Sampling)

Chain 1: Iteration: 400 / 500 [80%] (Sampling)

Chain 1: Iteration: 450 / 500 [90%] (Sampling)

Chain 1: Iteration: 500 / 500 [100%] (Sampling)

Chain 1:

Chain 1: Elapsed Time: 0.01 seconds (Warm-up)

Chain 1: 0.003 seconds (Sampling)

Chain 1: 0.013 seconds (Total)

Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 7e-06 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.07 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 500 [0%] (Warmup)

```

Chain 2: Iteration:  50 / 500 [ 10%] (Warmup)
Chain 2: Iteration: 100 / 500 [ 20%] (Warmup)
Chain 2: Iteration: 150 / 500 [ 30%] (Warmup)
Chain 2: Iteration: 200 / 500 [ 40%] (Warmup)
Chain 2: Iteration: 250 / 500 [ 50%] (Warmup)
Chain 2: Iteration: 251 / 500 [ 50%] (Sampling)
Chain 2: Iteration: 300 / 500 [ 60%] (Sampling)
Chain 2: Iteration: 350 / 500 [ 70%] (Sampling)
Chain 2: Iteration: 400 / 500 [ 80%] (Sampling)
Chain 2: Iteration: 450 / 500 [ 90%] (Sampling)
Chain 2: Iteration: 500 / 500 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 0.006 seconds (Warm-up)
Chain 2:                0.003 seconds (Sampling)
Chain 2:                0.009 seconds (Total)
Chain 2:

```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).

```

Chain 3:
Chain 3: Gradient evaluation took 5e-06 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.05 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:   1 / 500 [  0%] (Warmup)
Chain 3: Iteration:  50 / 500 [ 10%] (Warmup)
Chain 3: Iteration: 100 / 500 [ 20%] (Warmup)
Chain 3: Iteration: 150 / 500 [ 30%] (Warmup)
Chain 3: Iteration: 200 / 500 [ 40%] (Warmup)
Chain 3: Iteration: 250 / 500 [ 50%] (Warmup)
Chain 3: Iteration: 251 / 500 [ 50%] (Sampling)
Chain 3: Iteration: 300 / 500 [ 60%] (Sampling)
Chain 3: Iteration: 350 / 500 [ 70%] (Sampling)
Chain 3: Iteration: 400 / 500 [ 80%] (Sampling)
Chain 3: Iteration: 450 / 500 [ 90%] (Sampling)
Chain 3: Iteration: 500 / 500 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 0.012 seconds (Warm-up)
Chain 3:                0.005 seconds (Sampling)
Chain 3:                0.017 seconds (Total)
Chain 3:

```

Look at the summary

```
fit
```

Inference for Stan model: anon_model.

3 chains, each with iter=500; warmup=250; thin=1;

post-warmup draws per chain=250, total post-warmup draws=750.

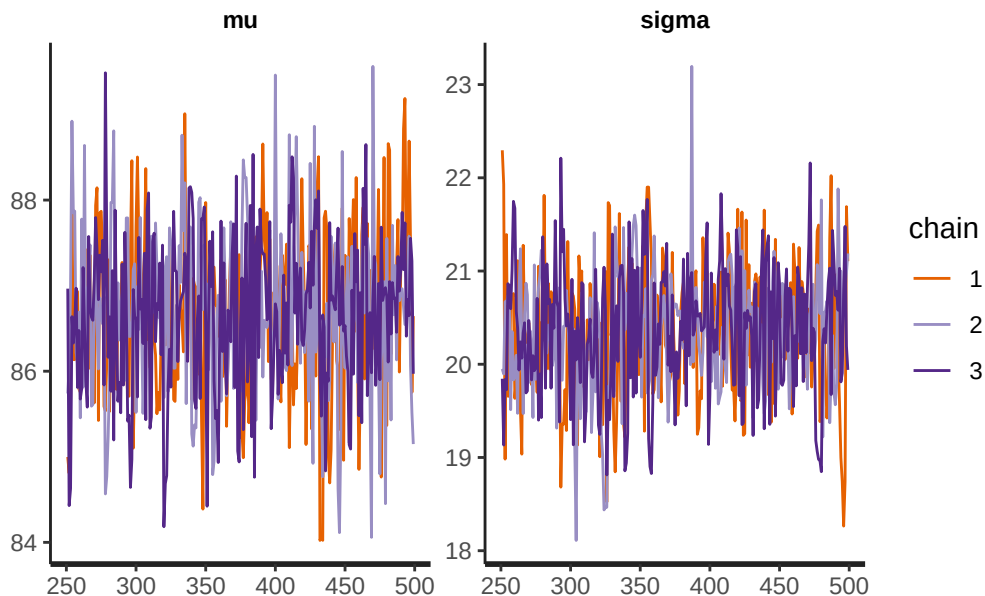
	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff
mu	86.68	0.04	0.94	84.77	86.04	86.72	87.32	88.54	561
sigma	20.35	0.03	0.68	18.99	19.87	20.37	20.82	21.66	486
lp__	-1525.72	0.05	1.02	-1528.70	-1526.04	-1525.37	-1525.03	-1524.77	378
Rhat									
mu	1								
sigma	1								
lp__	1								

Samples were drawn using NUTS(diag_e) at Fri Feb 6 08:48:19 2026.

For each parameter, n_eff is a crude measure of effective sample size, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat=1).

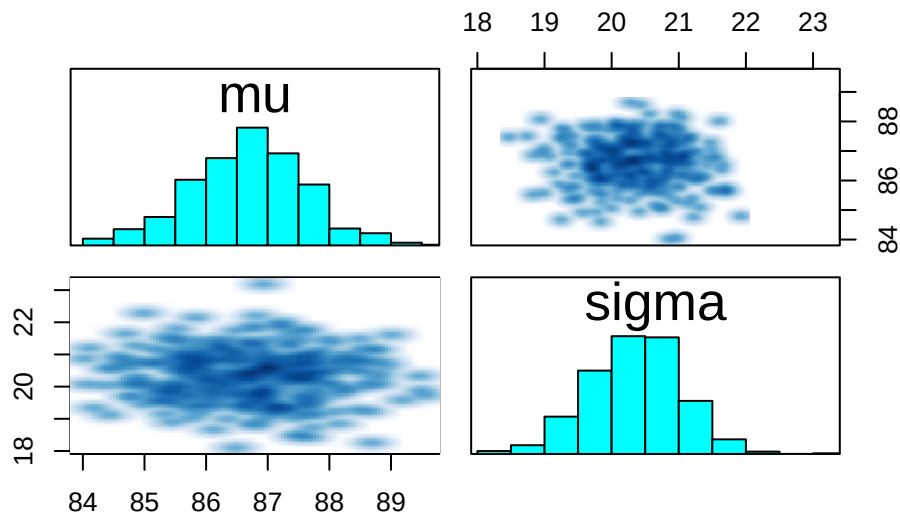
Traceplot

```
traceplot(fit)
```

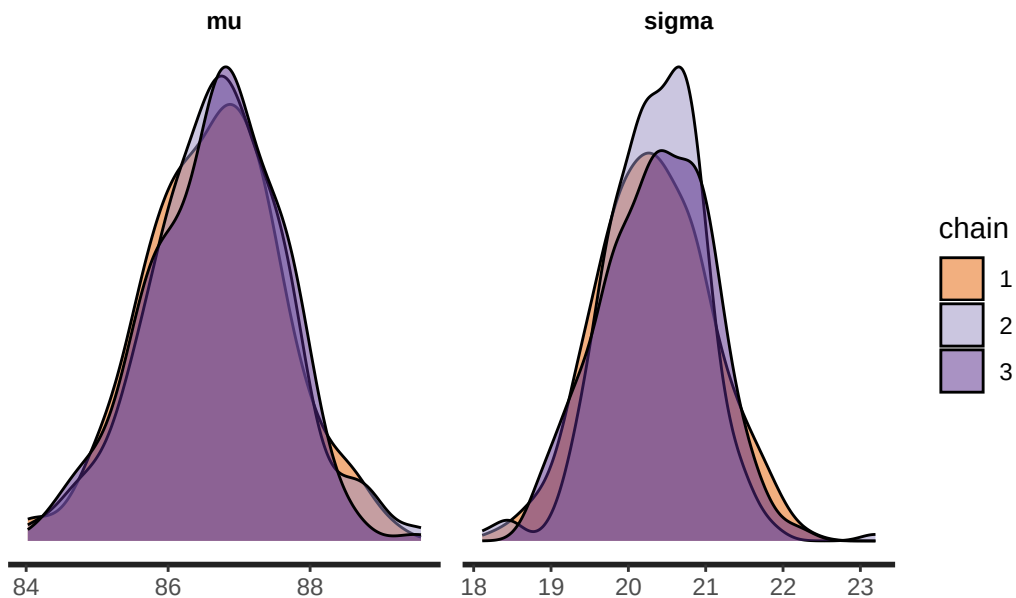


All looks fine.

```
pairs(fit, pars = c("mu", "sigma"))
```



```
stan_dens(fit, separate_chains = TRUE)
```



Understanding output

What does the model actually give us? A number of samples from the posteriors. To see this, we can use `extract` to get the samples.

```
post_samples <- extract(fit)
names(post_samples)
```

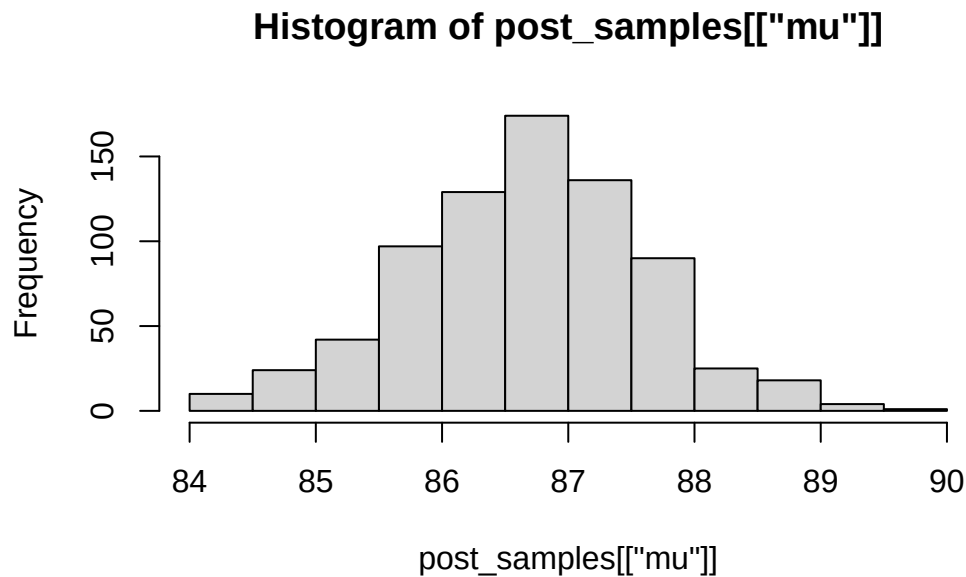
```
[1] "mu"      "sigma" "lp__"
```

```
head(post_samples[["mu"]])
```

```
[1] 87.15376 86.99230 86.77478 86.32321 86.38735 86.15523
```

This is a list, and in this case, each element of the list has 4000 samples. E.g. quickly plot a histogram of mu

```
hist(post_samples[["mu"]])
```



```
median(post_samples[["mu"]])
```

```
[1] 86.71621
```

```
# 95% bayesian credible interval
quantile(post_samples[["mu"]], 0.025)
```

```
2.5%
84.7696
```

```
quantile(post_samples[["mu"]], 0.975)
```

```
97.5%  
88.54263
```

Tidybayes is also very useful:

```
fit |>  
  gather_draws(mu, sigma) |>  
  median_qi(.width = 0.8)
```

```
# A tibble: 2 x 7  
  .variable .value .lower .upper .width .point .interval  
  <chr>      <dbl> <dbl> <dbl> <dbl> <chr> <chr>  
1 mu          86.7  85.5  87.8  0.8 median qi  
2 sigma       20.4  19.5  21.2  0.8 median qi
```

Plot estimates

There are a bunch of packages, built-in functions that let you plot the estimates from the model, and I encourage you to explore these options (particularly in `bayesplot`, which we will most likely be using later on). I like using the `tidybayes` package, which allows us to easily get the posterior samples in a tidy format (e.g. using `gather_draws` to get in long format). Once we have that, it's easy to just pipe and do `ggplots` as usual.

Get the posterior samples for `mu` and `sigma` in long format:

```
dsamples <- fit |>  
  gather_draws(mu, sigma) # gather = long format  
dsamples
```

```
# A tibble: 1,500 x 5  
# Groups:   .variable [2]  
  .chain .iteration .draw .variable .value  
  <int>    <int> <int> <chr>    <dbl>  
1      1        1     1 1 mu      85.0  
2      1        2     2 2 mu      84.8  
3      1        3     3 3 mu      85.6  
4      1        4     4 4 mu      88.0  
5      1        5     5 5 mu      86.1
```



```

6      1      6      6 mu      86.1
7      1      7      7 mu      86.9
8      1      8      8 mu      86.9
9      1      9      9 mu      85.8
10     1     10     10 mu      85.8
# i 1,490 more rows

```

```

# wide format
fit |> spread_draws(mu, sigma)

```

```

# A tibble: 750 x 5
  .chain .iteration .draw    mu sigma
  <int>     <int> <int> <dbl> <dbl>
1       1         1     1  85.0  22.3
2       1         2     2  84.8  21.9
3       1         3     3  85.6  19.0
4       1         4     4  88.0  21.4
5       1         5     5  86.1  19.7
6       1         6     6  86.1  19.7
7       1         7     7  86.9  20.7
8       1         8     8  86.9  20.7
9       1         9     9  85.8  20.8
10      1        10    10  85.8  19.6
# i 740 more rows

```

```

# quickly calculate the quantiles using

```

```

dsamples |>
  median_qi(.width = 0.8)

```

```

# A tibble: 2 x 7
  .variable .value .lower .upper .width .point .interval
  <chr>     <dbl> <dbl> <dbl> <dbl> <chr> <chr>
1 mu       86.7  85.5  87.8   0.8 median qi
2 sigma    20.4  19.5  21.2   0.8 median qi

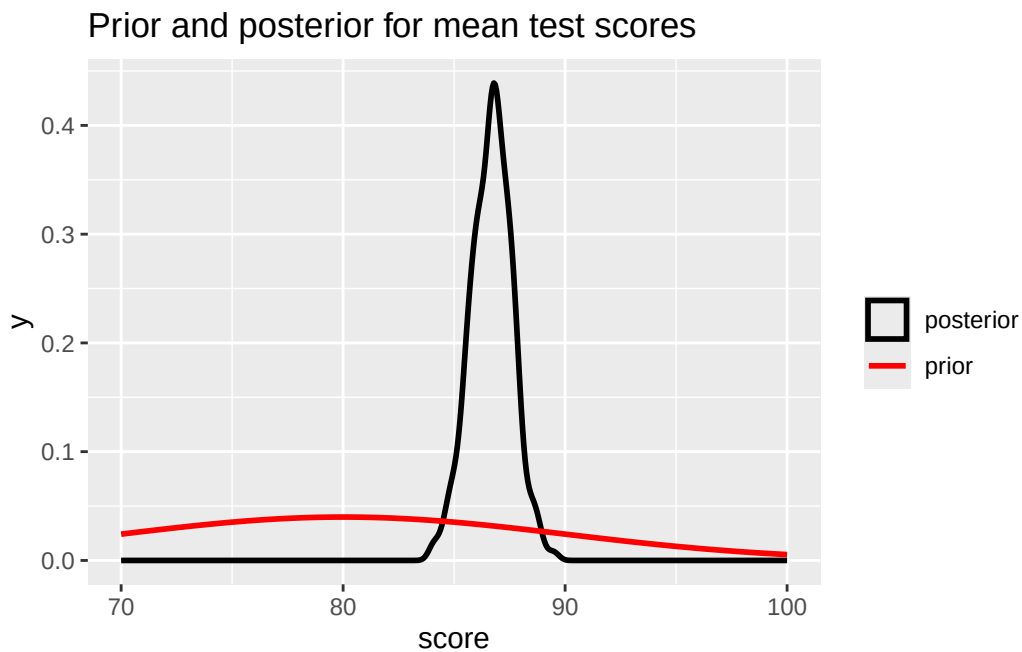
```

Let's plot the density of the posterior samples for mu and add in the prior distribution

```

dsamples |>
  filter(.variable == "mu") |>
  ggplot(aes(.value, color = "posterior")) + geom_density(size = 1) +
  xlim(c(70, 100)) +
  stat_function(fun = dnorm,
               args = list(mean = mu0,
                           sd = sigma0),
               aes(colour = 'prior'), size = 1) +
  scale_color_manual(name = "", values = c("prior" = "red", "posterior" = "black")) +
  ggtitle("Prior and posterior for mean test scores") +
  xlab("score")

```



Question 2

Change the prior to be much more informative (by changing the standard deviation to be 0.1). Rerun the model. Do the estimates change? Plot the prior and posterior densities.

Adding covariates

Now let's see how kid's test scores are related to mother's education. We want to run the simple linear regression

$$y_i | \mu_i, \sigma^2 \sim N(\mu_i, \sigma^2)$$

$$\mu_i = \alpha + \beta X_i$$

Priors:

$$\alpha \sim N(0, 100^2)$$

$$\beta \sim N(0, 10^2)$$

$$\sigma \sim N(0, 10^2)$$

where $X = 1$ if the mother finished high school and zero otherwise.

`kid3.stan` has the stan model to do this. Notice now we have some inputs related to the design matrix X and the number of covariates (in this case, it's just 1).

Let's get the data we need and run the model.

```
X <- as.matrix(kidiq$mom_hs, ncol = 1) # force this to be a matrix
K <- 1

data <- list(y = y, N = length(y),
             X = X, K = K)
fit2 <- stan(file = here("code/models/kids3.stan"),
             data = data,
             iter = 1000)
```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 4.3e-05 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.43 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 1000 [0%] (Warmup)

Chain 1: Iteration: 100 / 1000 [10%] (Warmup)

Chain 1: Iteration: 200 / 1000 [20%] (Warmup)

Chain 1: Iteration: 300 / 1000 [30%] (Warmup)

Chain 1: Iteration: 400 / 1000 [40%] (Warmup)

Chain 1: Iteration: 500 / 1000 [50%] (Warmup)

Chain 1: Iteration: 501 / 1000 [50%] (Sampling)

Chain 1: Iteration: 600 / 1000 [60%] (Sampling)

Chain 1: Iteration: 700 / 1000 [70%] (Sampling)
Chain 1: Iteration: 800 / 1000 [80%] (Sampling)
Chain 1: Iteration: 900 / 1000 [90%] (Sampling)
Chain 1: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 0.099 seconds (Warm-up)
Chain 1: 0.071 seconds (Sampling)
Chain 1: 0.17 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).

Chain 2:
Chain 2: Gradient evaluation took 1.6e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.16 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration: 1 / 1000 [0%] (Warmup)
Chain 2: Iteration: 100 / 1000 [10%] (Warmup)
Chain 2: Iteration: 200 / 1000 [20%] (Warmup)
Chain 2: Iteration: 300 / 1000 [30%] (Warmup)
Chain 2: Iteration: 400 / 1000 [40%] (Warmup)
Chain 2: Iteration: 500 / 1000 [50%] (Warmup)
Chain 2: Iteration: 501 / 1000 [50%] (Sampling)
Chain 2: Iteration: 600 / 1000 [60%] (Sampling)
Chain 2: Iteration: 700 / 1000 [70%] (Sampling)
Chain 2: Iteration: 800 / 1000 [80%] (Sampling)
Chain 2: Iteration: 900 / 1000 [90%] (Sampling)
Chain 2: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 0.115 seconds (Warm-up)
Chain 2: 0.066 seconds (Sampling)
Chain 2: 0.181 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).

Chain 3:
Chain 3: Gradient evaluation took 1.4e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.14 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration: 1 / 1000 [0%] (Warmup)

```

Chain 3: Iteration: 100 / 1000 [ 10%] (Warmup)
Chain 3: Iteration: 200 / 1000 [ 20%] (Warmup)
Chain 3: Iteration: 300 / 1000 [ 30%] (Warmup)
Chain 3: Iteration: 400 / 1000 [ 40%] (Warmup)
Chain 3: Iteration: 500 / 1000 [ 50%] (Warmup)
Chain 3: Iteration: 501 / 1000 [ 50%] (Sampling)
Chain 3: Iteration: 600 / 1000 [ 60%] (Sampling)
Chain 3: Iteration: 700 / 1000 [ 70%] (Sampling)
Chain 3: Iteration: 800 / 1000 [ 80%] (Sampling)
Chain 3: Iteration: 900 / 1000 [ 90%] (Sampling)
Chain 3: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 0.128 seconds (Warm-up)
Chain 3:                0.086 seconds (Sampling)
Chain 3:                0.214 seconds (Total)
Chain 3:

```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).

```

Chain 4:
Chain 4: Gradient evaluation took 1.6e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.16 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:   1 / 1000 [  0%] (Warmup)
Chain 4: Iteration: 100 / 1000 [ 10%] (Warmup)
Chain 4: Iteration: 200 / 1000 [ 20%] (Warmup)
Chain 4: Iteration: 300 / 1000 [ 30%] (Warmup)
Chain 4: Iteration: 400 / 1000 [ 40%] (Warmup)
Chain 4: Iteration: 500 / 1000 [ 50%] (Warmup)
Chain 4: Iteration: 501 / 1000 [ 50%] (Sampling)
Chain 4: Iteration: 600 / 1000 [ 60%] (Sampling)
Chain 4: Iteration: 700 / 1000 [ 70%] (Sampling)
Chain 4: Iteration: 800 / 1000 [ 80%] (Sampling)
Chain 4: Iteration: 900 / 1000 [ 90%] (Sampling)
Chain 4: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 0.125 seconds (Warm-up)
Chain 4:                0.074 seconds (Sampling)
Chain 4:                0.199 seconds (Total)
Chain 4:

```

```
fit2
```

Inference for Stan model: anon_model.

4 chains, each with iter=1000; warmup=500; thin=1;

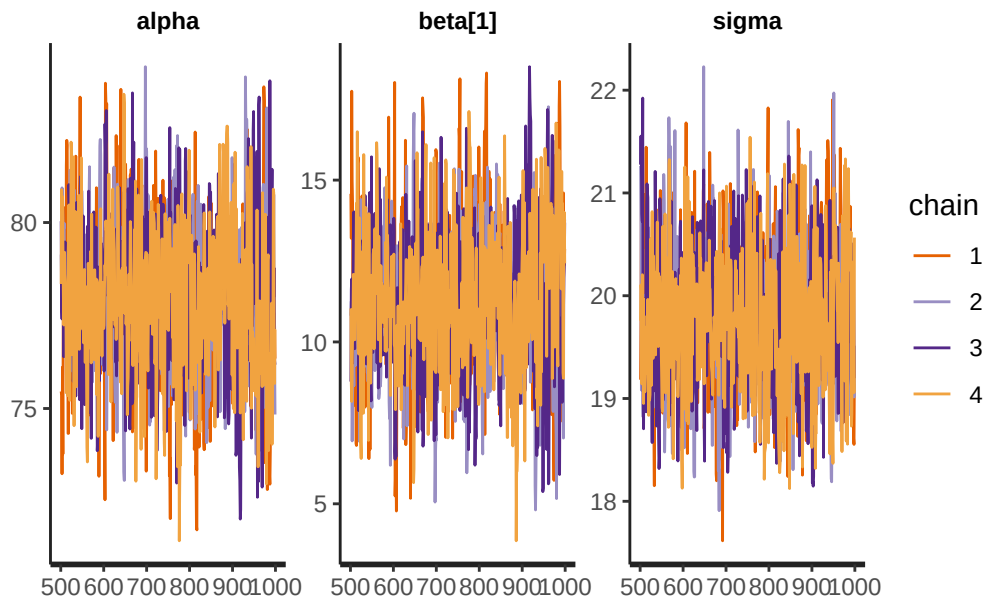
post-warmup draws per chain=500, total post-warmup draws=2000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%
alpha	77.93	0.07	2.01	73.90	76.61	77.94	79.30	81.97
beta[1]	11.29	0.08	2.24	6.94	9.80	11.24	12.76	15.93
sigma	19.80	0.02	0.69	18.48	19.31	19.77	20.28	21.18
lp__	-1514.40	0.04	1.23	-1517.61	-1514.99	-1514.05	-1513.52	-1512.99
	n_eff	Rhat						
alpha	821	1.00						
beta[1]	784	1.00						
sigma	980	1.00						
lp__	827	1.01						

Samples were drawn using NUTS(diag_e) at Fri Feb 6 08:49:23 2026.

For each parameter, n_eff is a crude measure of effective sample size, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat=1).

```
traceplot(fit2)
```



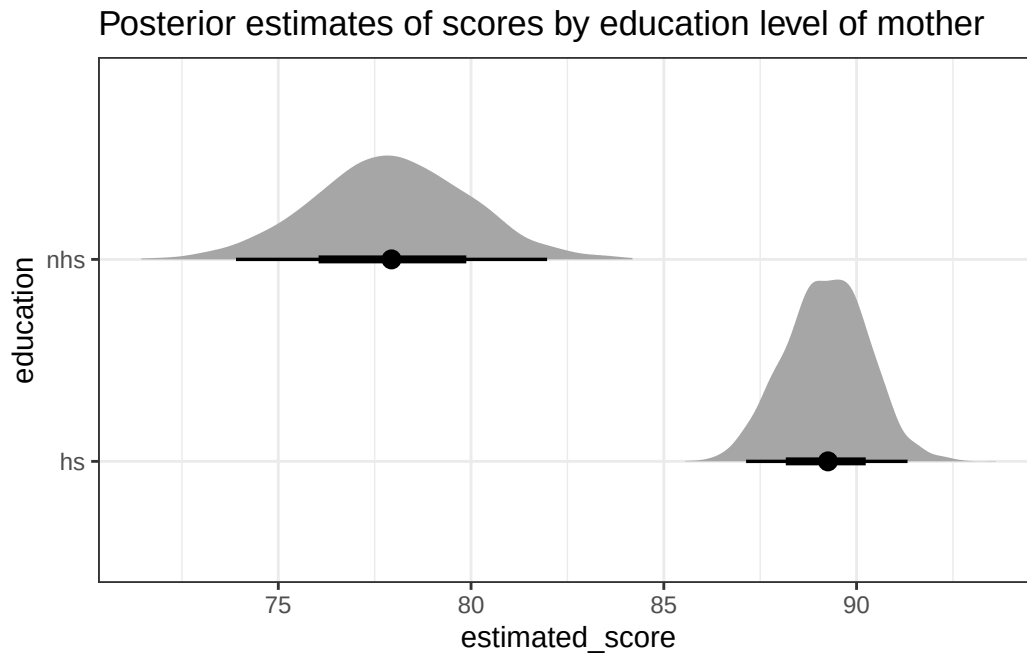
Question 3

- a) Confirm that the estimates of the intercept and slope are comparable to results from `lm()`
- b) Do a `pairs` plot to investigate the joint sample distributions of the slope and intercept. Comment briefly on what you see. Is this potentially a problem?

Plotting results

It might be nice to plot the posterior samples of the estimates for the non-high-school and high-school mothered kids. Here's some code that does this: notice the `beta[condition]` syntax. Also notice I'm using `spread_draws`, because it's easier to calculate the estimated effects in wide format

```
fit2 |>
  spread_draws(alpha, beta[k], sigma) |>
  mutate(nhs = alpha, # no high school is just the intercept
         hs = alpha + beta) |>
  select(nhs, hs) |>
  pivot_longer(nhs:hs, names_to = "education", values_to = "estimated_score") |>
  ggplot(aes(y = education, x = estimated_score)) +
  stat_halfeye() +
  theme_bw() +
  ggtitle("Posterior estimates of scores by education level of mother")
```



Question 4

Add in mother's IQ as a covariate and rerun the model. Please mean center the covariate before putting it into the model. Interpret the coefficient on the (centered) mum's IQ.

Question 5

Confirm the results from Stan agree with `lm()`

Question 6

Plot the posterior estimates of scores by education of mother for mothers who have an IQ of 110.

Question 7

Generate and plot (as a histogram) samples from the posterior predictive distribution for a new kid with a mother who graduated high school and has an IQ of 95.